Simulación de Memoria Virtual con LMDB

Proyecto N.° 13 — Arquitectura del Computador

Jesús Alberto Mora Lozada
C.I.: 28.480.557

Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología

Octubre de 2026

Contenido

1. El Problema
2. Objetivos
3. Técnicas Computacionales
4. Diseño del sistema
5. Metodología de medición
6. Análisis y Comparación de Resultados
7. Conclusiones

El Problema

El enunciado pide estudiar una técnica de los capítulos de **Memoria y Discos**, implementarla en C++ y **comparar el rendimiento** en distintos escenarios.

Proyecto N.° 13: usar **LMDB** para gestionar grandes volúmenes de datos como si fuera **memoria secundaria**.

La pregunta que se busca responder

Cuando los datos no caben en RAM, ¿cuánto cuesta *realmente* traerlos del disco? Esa diferencia es la **paginación por demanda** de la memoria virtual.

Dominio elegido: apuestas de lotería venezolana — registros pequeños y numerosos, con consultas puntuales y por rango (ideal para un B+tree).

Objetivos

General: implementar un sistema en C++ sobre LMDB que gestione grandes volúmenes de datos como memoria secundaria, mida la latencia y demuestre la paginación por demanda.

Específicos:

- Diseñar claves binarias que exploten el orden del B+tree (big-endian, índices secundarios, `MDB_APPEND`).
- Construir un generador determinista de datasets a escala.
- Medir escritura, latencia y **fallos de página** mayor/menor.
- Comparar acceso con **caché tibia** (RAM) vs **fría** (disco).
- Validar los agregados contra un manifiesto de verdad absoluta.

Técnica 1: memoria virtual y `mmap`

- El SO divide la memoria en **páginas** (4 KB).
- Al acceder a una página ausente ocurre un **fallo de página**: el kernel la trae del disco.
- **Mayor** = I/O real al dispositivo; **menor** = página ya en RAM.
- Traducción virtual→física: tabla de páginas + **TLB**.

Fuente: Patterson y Hennessy, §5.4 (pág. 492), §5.2–5.3.

`mmap`

LMDB **mapea el archivo** al espacio de direcciones. Leer una clave es leer *memoria*: el **caché de páginas del SO ES el caché**.

Técnica 2: LMDB y el árbol B+

LMDB es una base **clave-valor** (diccionario en disco):

- Un archivo mapeado a memoria.
- **Sin caché propio** (usa el del SO).
- B+tree copia-en-escritura, MVCC.
- **Un solo escritor** a la vez.

Árbol B+ (índice hacia el disco):

- Nodos = páginas con **abanico alto**.
- 3–4 niveles indexan miles de millones de claves.
- **Hojas enlazadas** ⇒ barridos por rango.

Búsqueda en 3–4 páginas, no en decenas.

Técnica 3: claves big-endian

LMDB compara claves con `memcmp` (bytes crudos). Si un entero va en **little-endian**, el orden de bytes **no** coincide con el orden numérico y los rangos fallan.

Regla del proyecto

Toda clave con enteros multi-byte va en **BIG-endian**: así el orden lexicográfico de bytes *es* el orden numérico.

Clave `bets` (13 B)	lotería u8 — ts u64 (BE) — taquilla u16 — seq u16

Diseño: sub-bases e índices

Un entorno LMDB con **cinco sub-bases** (una por patrón de consulta):

Base	Para qué
`bets`	registro maestro (por fecha)
`i_animalito`	frecuencia y rachas por figura
`i_terminal`	frecuencia por terminación
`i_taquilla`	replay por punto de venta
`draws`	resultados de los sorteos

Cada jugada se escribe en la base maestra y en los índices que le tocan.

Metodología: tibio vs frío

- **Tibio:** las páginas están en RAM (caché de páginas).
- **Frío:** se liberan las páginas antes de medir.

Caché fría sin privilegios de root

`madvise(MADV_DONTNEED)` sobre la región `mmap` del archivo (localizada en `/proc/self/maps`). La alternativa clásica (`echo 3 > /proc/sys/vm/drop_caches`) exige root.

Fallos de página: diferencia de los contadores `minflt` y `majflt` de `/proc/self/stat` antes y después de cada corrida.

Resultado 1: escritura ordenada vs aleatoria

Métrica	**Append**	**Aleatorio**
Registros/s	1,94 M	226 K
Commit p50	11,6 ms	101 ms
Tamaño final	118 MB	398 MB

Lección

Ordenar las claves da $\times$**8,6** de rendimiento y $\times$**3,4** de espacio: cada inserción cae en el borde derecho del B+tree (`MDB_APPEND`) en vez de dividir páginas dispersas.

Resultado 2: escritura concurrente (taquillas)

Taquillas	Throughput	Commit p50	Degradación
1	1,34 M reg/s	11,2 ms	—
8	1,00 M reg/s	24,3 ms	$\times$0,75
16	943 K reg/s	26,3 ms	$\times$0,70

Cuello de botella

LMDB admite **un solo escritor**. Añadir taquillas no acelera: es la **ley de Amdahl** (§1.8). Solución real: *particionar* los datos, no añadir hilos.

Resultado 3: el costo de la paginación (10M)

Modo	Tibio	Frío	Factor
read-lookup (200 k)	907 K ops/s	3,4 K ops/s	$\times$267
read-scan (1,67 M)	2,0 M ops/s	1,4 M ops/s	$\times$1,4

Evidencia

El modo frío generó **83 251 fallos de página mayores** (I/O real), frente a **0** en tibio. Los barridos secuenciales sobreviven gracias a la **relectura anticipada** del kernel.

Resultado 4: escalamiento a 100 millones

- **100 000 000** de jugadas generadas.
- **15,5 GB** en disco (≈163 B/jugada).
- Agregados validados **149/149**.
- B+tree de **4 niveles** para `bets`.

Escala en disco

1 M	180 MB
10 M	1,6 GB
100 M	15,5 GB

Resultado 5: optimizar también es *no* interferir

Se probaron *hints* al kernel:

- `MADV_RANDOM` antes de lookups $\Rightarrow$ los **empeoró** $\times$**6,3** (desactiva la relectura anticipada que ayudaba).
- `MADV_SEQUENTIAL` antes de barridos $\Rightarrow$ mejora marginal ($+12\,\%$).

Conclusión

No interferir con las heurísticas del kernel **es en sí mismo una optimización**.

Conclusiones

- El dominio de apuestas es ideal: registros pequeños, ordenables y con consultas de negocio reales.
- El diseño de claves big-endian habilita `MDB_APPEND` y los barridos por rango ($\times$8,6).
- Los benchmarks confirman la teoría: los accesos aleatorios son $\times$**267** más lentos en frío; los secuenciales apenas se afectan.
- **83 251 fallos de página mayores** medidos: la paginación por demanda existe y se cuantifica.
- LMDB tiene **un solo escritor**: para escalar a miles de taquillas hay que *particionar*, no añadir hilos.
- El proyecto se validó a escala real: **100 M de jugadas** (15,5 GB) con 149/149 agregados exactos.

Referencias

- Patterson, D. A. y Hennessy, J. L. (2014). *Estructura y diseño de computadores: la interfaz hardware/software* (4.ª ed.). Reverté. Capítulos 5 (§5.2–5.4) y 6 (§6.3–6.7).
- OpenLDAP Foundation. *LMDB: Lightning Memory-Mapped Database*. `https://www.lmdb.tech/doc/`
- `mmap(2)`, `madvise(2)` — páginas del manual de Linux.

Repositorio: `https://github.com/jmxsdev/simulacion-memoria-virtual-lmdb`

Gracias

Preguntas y discusión